



Act 1.1 Algorithm Design Mini Lab

Andrea Villaseñor Jimenez A01642088

Analysis and Design of Advanced Algorithms

Grupo 601

Gildardo Sanchez-Ante

Tuesday, August 29th, 2026

Act 1.1 Algorithm Design Mini Lab

For this activity, we shall choose between the following algorithms. To implement correct and efficient solutions.

1. *Divide & conquer* - Needs different subproblems (fragments) and may have a recursive nature easy to spot. Mostly used to process data or give it structure,
2. *Greedy* - Will act fast taking decisions (most convenient for each case). Sometimes useful when in need of minimization/maximization (optimization).
3. *Dynamic programming* - Collisions while solving (need of memory). This is also often used for minimization and maximization (optimization).

A. Inversion Counter for Quality Control - Problem A: “An array A of n integers. A pair (i, j) is an inversion if $i < j$ and $A[i] > A[j]$. **Task:** Compute the number of inversions in the array”, (Sanchez-Ante, 2026)

Problem A	
Input	An integer n A list of n integers
Output	A single integer: the inversion count
Constraints	$1 \leq n \leq 2 * 10^5$ Values fit in 32-bit signed int Output may exceed 32-bit; use 64-bit integer.
Examples	Input: A = [2, 4, 1, 3, 5] Output: 3 (inversions: (2,1), (4,1), (4,3)) <u>Notes</u> A direct $O(n^2)$ approach will be too slow for the upper constraints. Your algorithm should scale to large n.

a. Explanation of my solution

i. Reasoning behind choices taken

Starting the analysis of the problem, we see hints to discard the use of dynamic programming. The overall nature of the problem doesn't need continuous repetitive calculations (thus, no need for extra memory). This will mean no overlap, since we will just need to create a counter for the inversions present in the list. Therefore, there is only a need to differentiate between the pros and cons of a *greedy algorithm* and *divide and conquer*. For example, we might see greedy algorithms act faster, yet we shall consider whether, given an instance of the problem, this algorithm may be inadequate. Let the case be such that the index whose inversion is at 0 passes by unnoticed until reaching position 9; a greedy algorithm couldn't keep track since it can only look at the adjacent positions. If you “modify it” to fix this, it will only *patch* it and end up comparing every element n times against the n elements. Thus ending with a quadratic Big O complexity of $O(n^2)$.

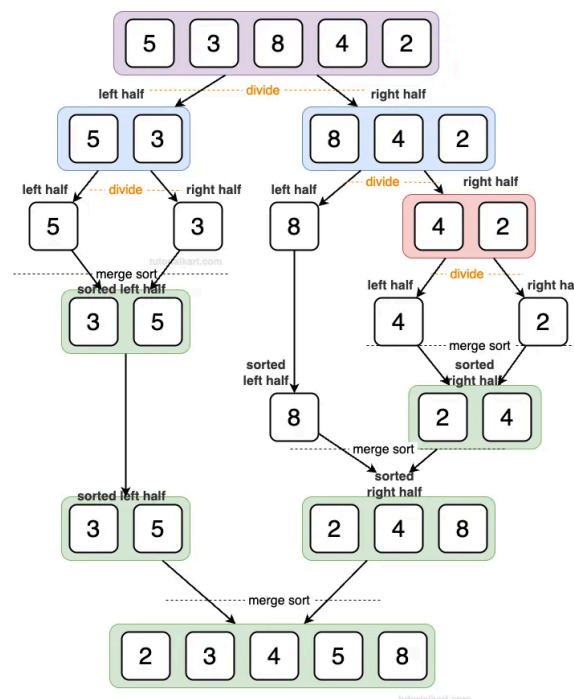
Meanwhile, with a *divide and conquer* approach, we may notice it is usually employed during *Merge Sort* and *Quick sort* (offering similar solutions to the problem's context). Now we could actually try to add a slight modification to these algorithms to count each alteration (order change) as an increase in the counter (inversions). Nevertheless, in the problem description there is no constraint nor restriction of spatial complexity, thus leaving my criteria to aim for *Merge Sort* rather than *Quick Sort*. This since in the worst (and best) case scenario of *Merge Sort* is $O(n \log n)$; yet if the list somehow turns out ordered (0 inventions) a *Quick Sort* instead of $O(n \log n)$, will have $O(n^2)$ computational complexity. Yet, it is crucial to note that Merge Sort has a spatial complexity of $O(n)$.

For further optimization, this algorithm is usually implemented with a data structure commonly known as a *Linked List*. If not used, when trying to unify both halves of the array, the need for additional memory will give it the complexity of $O(n)$, as previously mentioned.

Due to the elements in a Linked List being connected by pointers, when unifying, you just have to redirect pointers without an external copy or additional temporary space. Leading to a huge impact by decreasing the space complexity from $O(n)$ to $O(1)$. Meeting the standards of the note at problem A, proving the solution will be scalable even when working with a big value of n .

ii. Overall logic of my solution

A number n is received and interpreted as the number of elements in the array. The next input is the n numbers separated by spaces. There is no information as to whether there should be a validation that the quantity of elements separated by spaces corresponds to the first input (yet, we will assume we must validate). Then this will be represented by a list which will be divided into half ($n/2$), and both halves will be treated as a recursion, getting divided into halves as well until it gets every element individually. To later order it, getting the subarray from the left and right halves. Finally, everything gets integrated into one list only. Just as the following image illustrates.



Merge Sort (Tutorial Kart, n.d).

This is the overall behaviour of *Merge Sort* itself. We will add a counter such that it increments with every “exchange” of elements it has to do while ordering (inversions become the changes done to order it properly). To comply with the specifications. Showing in the terminal the total number of inversions found in the given array. With the use of this theoretical approach, we are using everything to our advantage to attack the computational complexity of $O(n^2)$.

b. Pseudocode

None

//Pseudocode

Linked List Operations

A. Function insertLast(data):

```
Create a new node 'newNode' with 'data'
If the list isEmpty:
    head = newNode
    tail = newNode
Else:
    tail.next = newNode
    tail = newNode
Increase list size
```

B. Function getMiddleNode(head):

```
If list has 0 or 1 element, return head
slow = head
fast = head.next
While fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
midNext = slow.next
slow.next = null // Disconnection of pointers: two independent halves
return midNext
```

Main Algorithm

1. Receive number n , which represents the total amount of elements.
 $n = \text{input}$
2. Receive the n numbers separated by spaces + validate that it matches n .
 $\text{input}, \text{input}, \dots, \text{nth_input}$
3. Linked List (while reading: no intermediate arrays)
 Call insertLast(input) for each element read
 $\text{input} \rightarrow \text{input} \rightarrow \dots \rightarrow \text{nth_input}$
4. Initialize global counter (for inversions)

```
counter_inversions = 0
```

Merge sort

5. If list >1 element: divide in half (recursive)

```
if (head == null or head.next == null) return head
mid = getMiddleNode(head) // Slow and fast pointers: to find the center
left = head
right = mid // Disconnection already handled in getMiddleNode
leftSorted = mergeSort(left)
rightSorted = mergeSort(right)
return merge(leftSorted, rightSorted)
```

6. If left element > right element -> increase counter (+1): (+ length(left)) merge(left, right):

6.1 Create a dummyNode to build the new merged list, and set current = dummyNode.

```
while (left != null and right != null):
    if (left.value <= right.value):
        current.next = left
        left = left.next
    else (Checks inversion):
        current.next = right
        right = right.next
    //Update counter:
    counter_inversions = counter_inversions + length(left) // All
remaining nodes in 'left' are greater than this 'right' node
```

7. Reorder (reintegration of list) -> end of merge & counter ready

```
//Link the remaining nodes from whichever list is not empty.
if (left != null): current.next = left
if (right != null): current.next = right
current = current.next // Advance pointer to the end
return dummyNode.next
```

8. Return counter

```
show(counter_inversions)
```

Note. The Merge Sort pseudocode was adapted from Section 5.6 Merge Sort (p. 328) of Estructuras de datos y algoritmos fundamentales (1st ed.), by V. M. de la Cueva Hernández, L. H. González Guerra, and E. Salinas Gurrión, 2020, Editorial Digital del Tecnológico de Monterrey.

Note. The base structure of the linked list and the generic Merge Sort algorithm presented in this code were adapted from the Integrative Activity 2.3 project developed for the course TC-1031.601 by K. D. Martínez Ponce, J. A. Márquez Martínez, and A. Villaseñor Jiménez (2025, Tecnológico de Monterrey).

- c. A worked example/test - 3 tests per problem (can be unit tests or input/output examples).

i. Test results 1

Problem A - test 1			
Input	6 2 4 2 5 3 4	Expected output	4

ii. Test results 2

Problem A - test 2			
Input	4 7 7 7 7	Expected output	0

iii. Test results 3

Problem A - test 3			
Input	5 10 20 30 40 1	Expected output	4

iv. Screenshots

Problem A	
Test 1	<pre> cpp -o prueba (base) andrea@MacBook-Air-de-Andrea-161 problemA_inversions % ./prueba 6 2 4 2 5 3 4 Inversions found: 0 </pre>
Test 2	<pre> (base) andrea@MacBook-Air-de-Andrea-161 problemA_inversions % ./prueba 4 7 7 7 7 Inversions found: 0 </pre>
Test 3	<pre> (base) andrea@MacBook-Air-de-Andrea-161 problemA_inversions % ./prueba 5 10 20 30 40 1 Inversions found: 0 </pre>

d. Detailed time/space complexity analysis

- Time Complexity: $O(n \cdot \log n)$. The algorithm recursively divides the linked list into halves using a slow and fast pointer approach, which takes $O(n)$ time per level. The division creates a tree with a depth of $\log n$. Unifying the split components in the merge step processes each node exactly once per level, taking $O(n)$ time. Therefore, the overall time complexity remains strictly bounded by $O(n \cdot \log n)$, ensuring efficient execution within the constraints of $n = 2 \times 10^5$.
- Space Complexity: $O(\log n)$ auxiliary space. Because the data is stored in a Linked List, the merge operation bypasses the need for temporary sub-arrays by simply redirecting node pointers in $O(1)$ space. While the primary data structure takes $O(n)$ space to store the inputs, the sole auxiliary space overhead stems from the recursive call stack. With a balanced recursive tree, the maximum memory depth is bounded to $\log n$.

- B. Minimum Meeting Rooms (Scheduling) - Problem B:** “You are given m meetings, each with a start time and an end time: (s_i, e_i) with $s_i < e_i$. A room can host *only one meeting at a time*. **Task:** Compute the minimum number of rooms required to schedule all meetings”, (Sanchez-Ante, 2026).

Problem B	
Input	Integer m Then m lines: start end
Output	Minimum number of rooms needed
Constraints	$1 \leq m \leq 2 \cdot 10^5$ Times are integers (e.g., minutes) in range $[0, 10^9]$
Examples	$(0, 30), (5, 10), (15, 20)$ Output: 2

	<u>Clarifications</u> If one meeting ends at time t and another starts at time t, they do not overlap (same room is allowed). <u>Notes</u> Focus on an efficient approach for large m.
--	---

a. Explanation of my solution

i. Reasoning behind choices taken

In this context, it's an optimization problem, since we need the minimum number of rooms. A *divide and conquer* approach won't help determine if there's a schedule overlap. Therefore, it will be discarded as it isn't the tailored solution. It is between *dynamic programming* and a *greedy algorithm*. Analyzing different instances of this particular problem is key to choosing. Now, since the *greedy algorithm* makes short-term fast decisions, we have to ask if it could harm the future outcome. However, it is irrelevant whether meeting 1 happens inside room 1, 2, or k . There doesn't need to be a thorough analysis of possibilities and combinations among meetings and rooms where they are held. This leaves *dynamic programming* out of the way for a *greedy algorithm* to succeed at completing the minimization task.

These meeting schedules could be ordered, perhaps by start time (ascending order). This could be made with the sort inside C++ (in the library *algorithm*), which is $O(m \log m)$. When you notice a meeting shall start, the first thing would be to look out for any room that is unoccupied, which you could assign immediately. If not, you assign another room (a new one). Since this is fast and the only thing you could do at that specific moment, you haven't the space for errors. Thus, there shall be no need to rewind previous decisions. Yet, if two meetings happen to have the same start time (or overlap in any way), it can complicate the overall logic and functionality of the arrangement if not foreseen correctly.

Leaving possible room for errors. Therefore, there has to be a memory of which room a meeting is being held in. This shall be done by integrating another data structure.

We can see that a *Queue* (FIFO) or a *Stack* (LIFO) isn't even an option, since it is not interpreting the data correctly. They are only relevant at the time of insertion, providing no further interpretation. Both data structures are too rigid and unable to track availability dynamically. We want this data to be ordered dynamically and prioritized. Therefore, a *Hash Table* is also discarded (since we would have to search through it, leading to an $O(m^2)$ complexity) because it doesn't order itself dynamically either. This leaves the possibility of naturally ordered structures, such as *trees* (e.g., *BSTs*). Yet, the tree has all elements ordered at all times and has pointers for nodes. Leaving space for a better data structure that manages memory more efficiently.

Nevertheless, we see that since there could be no loops, graphs are also discarded. Finally, we could analyze another structure that is similar to a *tree* (a *heap*). For instance, a *Min Heap* acts like a tree, yet needs no pointers to work. It is known as a *Priority Queue* (since it attends to the need that a normal *queue* disregards). Plus, the only strict rule it has benefits the solution thoroughly since the minimum number should always be in the first position. Therefore, consulting which meeting room is free is in $O(1)$, and further insertions or removals are in $O(\log k)$, where k is the number of meeting rooms simultaneously occupied, or the current size of the *Priority Queue*.

Now, taking these decisions into account, it is important to note something. The auxiliary space complexity of this should be $O(k)$, since the heap only stores the rooms (k) that are active at the same time, while the total space complexity including the input array is $O(m)$. This matters, since it gives more memory efficiency. Letting the overall solution be prepared for a large value of m .

ii. Overall logic of my solution

First, the program will have to read the data, since the *Min Heap* cannot process this. For which we could have an array (vector in C++) with the start and ending of the meetings (schedule). No further details as to a validation procedure are stated regarding the number m and the actual number of elements given. Nevertheless, there will be one just in case. Now that all meetings are sorted, we shall sort them out based on the starting time (using C++'s sort). Then, there has to be a *Min Heap*, so it will get created (empty for now). This is where the tricky part happens, because the *heap* will actually represent the list of occupied rooms. Yet, to the *heap* itself we will insert only the ending hour of each meeting. If by chance a room is unoccupied (meeting start time $\geq$ room liberation time), it will be popped from the *heap*. To know exactly when this room will be free.

The overall flow will be to analyze the meeting you want to schedule. Then, looking at the *root* of the *Min Heap* (first element), which stands for the first room's liberation. If by chance a room is unoccupied, it will get popped off the *heap*. Furthermore, we will add the ending of the meeting we analyzed to the *heap* (recycling a room). If no room has a vacancy, then a new room will be needed (insertion of the meeting analyzed, and no deletion of rooms inside the *heap*). Yet, since this structure's advantage relies on its contents being ordered, we must do something about it. Notice that this will happen automatically with the C++ *heap*, yet this is just to understand how this will all get ordered. This with the help of a *heapify-up* and *down* it will order itself. The *heapify-up* will be done when a new "room" is inserted into the heap, and a *heapify-down* will be used when removing the *root* element (which happens when a room becomes unoccupied) from the *heap*. Finally, once all meetings have been processed, the total count of elements remaining in the *heap* will represent the minimum number of rooms required, which is then returned as the result.

b. Pseudocode

None

//Pseudocode

1. Recieve number. Save it as m, which represents the total number of meetings.

m = input

2. Recieve 2 numbers spearated by a space (loop for as many meetings) + validate with m.

meetings_v = empty_list() for a lenght m: start, end =
read_two_integers()

3. Store pairs in vector

meetings_v.append((start, end))

4. Sort (ascending meetings' start times)

sort(meetings_v, based_on = start_time)

5. Initialize Min Heap

min_heap = empty_array()

6. Loop: to iterate through vector for meetings. For each meeting, look at the root of the Min Heap (room that has a vacancy the earliest).

for current_meeting in meetings_v:

6.1. If this room's end time is less than or equal to the current meeting's start time, the room is vacant. Pop it from the heap.

if (min_heap.length > 0) and (min_heap[0] <=
current_meeting.start): Heap_Pop(min_heap)

6.2. Insert (push) the current meeting's ending time into the Min Heap. This signifies that a room (recycled one/new one) is now occupied.

Heap_Push(min_heap, current_meeting.end)

7. End of loop. Min heap -> has total number of rooms needed (minimum). Return size/number of elemnts in the heap.

return min_heap.length

//Auxiliary Heap Operations Pseudocode -> done automatically by the C++ heap (yet just for visualization purposes)

#1 - Function Heap_Pop(heap_array):

1. Take the value of the last leaf of the HEAP, that is, the last element of the array.

root_value = heap_array[0]
last_index = heap_array.length - 1
last_leaf_value = heap_array[last_index]

2. Swap this value with the one at the root.

heap_array[0] = last_leaf_value

3. Physically eliminate the node that represents the last leaf.

heap_array.remove_at(last_index)

4. Finally, reorder downwards the element that remained as the root of the HEAP:

current = 0
while ((current * 2) + 1) < heap_array.length:

4.1. Compare the value of the node with the child nodes.

left_child = (current * 2) + 1

```

    right_child = (current * 2) + 2
    highest_priority_child = left_child
    if (right_child < heap_array.length and heap_array[right_child] >
        priority than heap_array[left_child]):
        highest_priority_child = right_child

```

4.1.1. If the value of the parent has a higher priority, the reordering process ends here.

```

    if (heap_array[current] >= priority than
        heap_array[highest_priority_child]):
        break

```

4.1.2. If the value of any child has a higher priority, swap this with the parent, and repeat the downward reordering process with the children of the swapped element.

```

        swap(heap_array[current], heap_array[highest_priority_child])
        current = highest_priority_child
    return root_value

```

#2 - Function Heap_Push(heap_array, new_element):

1. Insert the new element as the last leaf of the HEAP, that is, as the last element of the array.

```

    heap_array.append(new_element)
    current = heap_array.length - 1

```

2. Compare this new element with respect to its parent; if the element was inserted at position k, the parent will be at position k/2.

```

    // Adjusted for 0-index: parent is at (k-1)/2
    while (current > 0):
        parent = (current - 1) / 2

```

3. If the value of the parent has a lower priority, swap them and continue comparing upwards.

```

        if (heap_array[parent] < priority than heap_array[current]):
            swap(heap_array[parent], heap_array[current])
            current = parent

```

4. If the value of the parent has a higher priority, the process ends there.

```

    else:
        break

```

Note. The Heap pseudocode was adapted from Section 9.4 Árboles Heap (p. 523) of *Estructuras de datos y algoritmos fundamentales* (1st ed.), by V. M. de la Cueva Hernández, L. H. González Guerra, and E. Salinas Gurrión, 2020, Editorial Digital del Tecnológico de Monterrey.

- c. A worked example/test - 3 tests per problem (can be unit tests or input/output examples).

i. Test results 1

Problem B - test 1			
Input	4 2 10 2 5 2 8 5 8	Expected output	3

ii. Test results 2

Problem B - test 2			
Input	4 1 100 10 20 30 40 50 60	Expected output	2

iii. Test results 3

Problem B - test 3			
Input	3 100 200 10 50 30 80	Expected output	2

iv. Test results 4

Problem B - test 4			
Input	3 10 20 20 30 30 40	Expected output	1

v. Screenshots

Problem B	
Test 1	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemB_rooms % ./programa 4 2 10 2 5 2 8 5 8 3 </pre>
Test 2	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemB_rooms % ./programa 4 1 100 10 20 30 40 50 60 2 </pre>
Test 3	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemB_rooms % ./programa 3 100 200 10 50 30 80 2 </pre>
Test 4	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemB_rooms % ./programa 2 3 10 20 20 30 30 40 1 </pre>

d. Detailed time/space complexity analysis.

- Time Complexity:** $O(m \cdot \log m)$. Sorting the initial array of m meetings by their start times dictates a baseline computational cost of $O(m \cdot \log m)$. Next, the algorithm iterates through all m elements. For each meeting, the Min Heap evaluates room availability, potentially executing a Heap_Pop and a Heap_Push. Given that the heap stores a maximum of k concurrent meetings (where $k \leq m$), heap operations take $O(\log k)$ time. The loop's total time is $O(m \cdot \log k)$, making the overarching limit $O(m \cdot \log m)$.
- Space Complexity:** $O(m)$ overall space. The initial array mapping out the schedule inherently requires $O(m)$ memory. The secondary structure, the Min Heap,

dynamically stores the end times of active rooms. In a worst-case scenario where every single meeting overlaps simultaneously, the heap size k expands to equal m . The auxiliary space is therefore $O(k)$, scaling to a maximum ceiling of $O(m)$.

- C. **Budgeted Study Plan (0/1 Knapsack Variant - Problem C:** “You have n study modules. Each module i requires $\text{time}[i]$ hours and yields $\text{value}[i]$ points of learning benefit. You have a total available time budget T . Each module can be taken at most once. **Task:** Compute the maximum total value achievable without exceeding time T ”, (Sanchez-Ante, 2026).

Problem C	
Input	Integers n and T Then n lines: time value
Output	Maximum value achievable Constraints (suggested): $1 \leq n \leq 2000$ $1 \leq T \leq 5000$ $1 \leq \text{time}[i] \leq T$ $1 \leq \text{value}[i] \leq 10^6$
Examples	<u>Example</u> Modules: (time=3, value=4), (time=4, value=5), (time=7, value=10) with $T=10$ Output: 14 (take modules 1 and 3: time $3+7=10$, value $4+10=14$)
Notes	<u>Notes</u> A brute force subset search is infeasible. Your solution should handle the full constraints.

a. Explanation of my solution

i. Reasoning behind choices taken

Divide and Conquer cannot be used for this problem because the primary objective is not to sort the elements (neither by time nor by value), as the total sum of chosen values must be optimized without exceeding the time limit T . On the other hand, a Greedy algorithm is also inadequate; by making local, short-term decisions—such as picking items solely based on their value-to-time ratio. It risks sacrificing the optimal global combination, leading to premature errors. Therefore, Dynamic Programming is the optimal approach. Especially because there is a limit (time T), and the overall structure of the problem resembles the famous *knapsack problem*. Yielding a computational complexity of $O(n \cdot T)$ and a spatial complexity of $O(T)$. The overlapping that dynamic programming needs is found since there could be several possibilities to sum up the same amount of time without exceeding the time limit T . Once a calculation is done, it is stored for an $O(1)$ access next time you need it.

ii. Overall logic of my solution

Just like in the Knapsack problem, “where we need to maximize profit by selecting items within a weight limit. Here, we define our state using two parameters”, (GeeksforGeeks, 2024). This way we will have to evaluate whether we will pick or not the object (a module in this case scenario). Like the image below illustrates (the Bottom-Up DP approach with space optimization). Where the formula for calculations is:

$$dp[w] = \max(dp[w], dp[w - time[i]] + value[i])$$

- w is the weight: in our context the current time capacity.
- i : current study module to be analyzed
- $time[i]$: time required to complete the current module i .
- $value[i]$: the points (learning benefit) yielded by the current module i .

Note. In this case, $dp[t] = \max(dp[t], dp[t - \text{time}[i]] + \text{value}[i])$ with a time limit T .

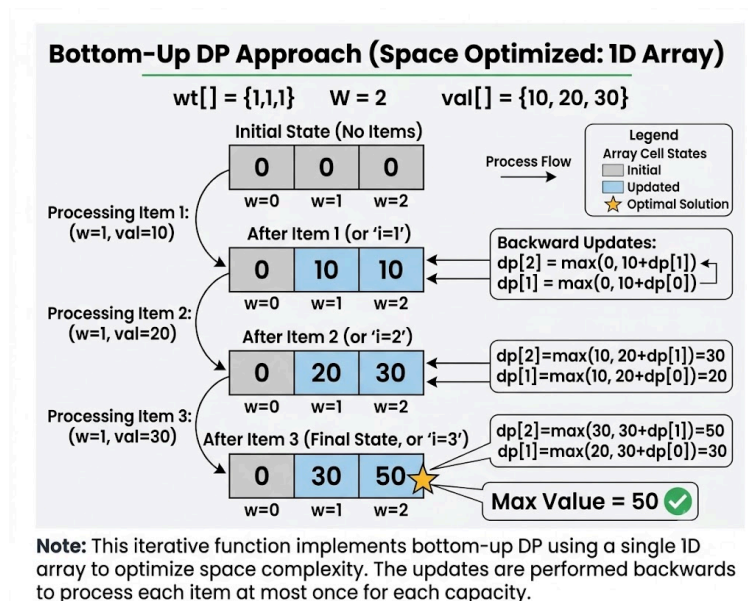


Imagen generada mediante la herramienta de inteligencia artificial Gemini (Google, 2026).

Se solicitó ilustrar la tabla de actualizaciones de un arreglo unidimensional para el problema de la mochila 0/1.

b. Pseudocode

None

//Pseudocode

1. Receive number. Save it as n , which represents the total number of study modules.

$n = \text{input}$

2. Receive number. Save it as max_t , which represents the maximum time budget.

$\text{max_t} = \text{input}$

3. Initialize a 1D DP array of size $\text{max_t} + 1$ with all its elements set to 0. Stored value: learning benefit (the optimal). Index: time capacity.

$\text{dp} = \text{empty_array}(\text{size} = \text{max_t} + 1, \text{fill} = 0)$

4. Outer loop: to receive and evaluate each module one by one on-the-fly, extracting its specific required time and yielded value without storing them in a separate data structure.

for i from 0 to $n - 1$:

$\text{current_time}, \text{current_value} = \text{read_two_integers}()$

5. Inner loop: DP backwards (from the absolute limit max_t down to the current module's required time).

for w from max_t down to current_time :

6. Update current position inside the inner loop.

```
skip_module = dp[w]
take_module = dp[w - current_time] + current_value
dp[w] = max(skip_module, take_module)
```

7. Once finished, the absolute maximum of learning benefit should be found at the last index (of the array). Return it.

```
return dp[max_t]
```

c. A worked example/test - 3 tests per problem (can be unit tests or input/output examples).

i. Test results 1

Problem C - test 1			
Input	3 6 3 10 3 10 3 10	Expected output	20

ii. Test results 2

Problem C - test 2			
Input	3 8 5 50 4 36 4 36	Expected output	72

iii. Test results 3

Problem C - test 3			
Input	4 10 10 100 2 30 3 40 5 50	Expected output	120

iv. Screenshots of test

Problem C	
Test 1	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa 3 6 3 10 3 10 3 10 20 </pre>
Test 2	<pre> 20 • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa 3 8 5 50 4 36 4 36 72 </pre>
Test 3	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa 4 10 10 100 2 30 3 40 5 50 120 </pre>
Alternative input	<pre> • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa < test1.txt 20 • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa < test2.txt 72 • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % ./programa < test3.txt 120 • (base) andrea@MacBook-Air-de-Andrea-161 problemC_knapsack % </pre>

d. Detailed time/space complexity analysis.

- Time Complexity:** $O(n \cdot T)$. The algorithm relies on a direct nested loop structure. The outer loop captures and evaluates each of the n study modules. For every module, the inner loop iterates backward from the time budget ceiling T down to the current module's required time. Because the calculation at each step evaluates a simple $O(1)$ mathematical maximum, the total iterations compute strictly in $n \times T$ operations.
- Space Complexity:** $O(T)$ auxiliary space. Rather than applying a standard 2D matrix which would command an $O(n \cdot T)$ footprint, this implementation leverages space optimization via a singular 1D DP array sized $T + 1$. Because the required times and values are processed dynamically on-the-fly inside the loop without storing them in

independent arrays of size n , the memory consumption is strictly confined to the time limit array.

Note. The detailed time and space complexity analysis for Problems A, B, and C was drafted and formatted with the assistance of the Gemini language model (Google, 2026). The generated text is based strictly on the original C++ source code, the overall explanation of the report, and the algorithmic reasoning previously established in the project's README.

References

- Anthropic. (2026). Claude (Sonnet 5) [Large language model]. <https://claude.ai>*
- Chio Code. (2022, 29 de noviembre). Merge Sort | Ordenamiento por mezcla [Video].
YouTube. https://www.youtube.com/watch?v=ACFZn_xQcz8
- Chio Code. (2022, 6 de septiembre). Quick Sort | Ordenamiento Rápido | Explicado con
Cartitas! YouTube. <https://www.youtube.com/watch?v=UrPJLhKF1jY>
- De la Cueva Hernández, V. M., González Guerra, L. H., & Salinas Gurrión, E. (2020).
Estructuras de datos y algoritmos fundamentales (1.^a ed.). Editorial Digital del
Tecnológico de Monterrey.
- GeeksforGeeks. (2024). Solve Dynamic Programming Problem.
<https://www.geeksforgeeks.org/dsa/solve-dynamic-programming-problem/>
- GeeksforGeeks. (2026). *0-1 Knapsack Problem*.
<https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>
- GeeksforGeeks. (s.f.). Introduction to Heap.
<https://www.geeksforgeeks.org/dsa/introduction-to-heap/>
- Google. (2026). Gemini (Versión del 30 de agosto) [Modelo de lenguaje grande].
<https://gemini.google.com>
- Martínez Ponce, K. D., Márquez Martínez, J. A., & Villaseñor Jiménez, A. (2025). Actividad
integradora 2.3 [Código fuente]. Tecnológico de Monterrey.
- Sanchez-Ante, G. (2026). Act 1.1 Algorithm Design Mini Lab [Instrucciones de la tarea].
Canvas ITESM. <https://tedu.tec.mx/es/canvas>
- TutorialKart. (s.f.). Merge Sort Algorithm.
<https://www.tutorialkart.com/algorithms/merge-sort-algorithm/>